

ROUTEZONE

Prédiction de la gravité des accidents routiers

RAPPORT PROFESSIONNEL — ÉPREUVE E3

Intégration des modèles et des services d'intelligence artificielle

Candidate	Meriem Abdelouahed
Formation	Développeur en Intelligence Artificielle — Simplon × Microsoft
Certification	RNCP37827
Soutenance	22 juin 2026 — distanciel
Version	2.0 — 29 mai 2026

Sommaire

1. Introduction	4
2. Architecture globale du service IA	4
2.1 Vue d'ensemble	4
2.2 Flux de données pour une prédiction	5
2.3 Endpoints et documentation Swagger	5
2.4 Justification des choix d'architecture	5
3. API REST IA — FastAPI	5
3.1 Pourquoi FastAPI	5
3.2 Architecture interne de l'API	5
3.3 Liste des endpoints	6
3.4 Authentification : API Key + JWT	6
a) Authentification par clé API	6
b) Authentification par JWT (utilisateurs humains)	7
c) Hashage bcrypt des mots de passe	7
3.5 Validation des entrées avec Pydantic	7
3.6 Gestion des erreurs	7
3.7 Conformité RGPD	7
3.8 Sécurité OWASP	8
4. Modèle ML servi (LightGBM)	9
4.1 Rappel rapide du modèle	9
4.2 Chargement du modèle	9
4.3 Stratégie multi-versions	9
4.4 Pipeline de prédiction	9
4.5 Décision technique : calibrator désactivé	10
4.6 Performances en production	10
5. Application prototype Streamlit	10
5.1 Présentation de Streamlit	10
5.2 Utilisateurs cibles	10
5.3 Parcours utilisateur	10
5.4 Communication entre Streamlit et l'API	12
5.5 Design et identité visuelle	12
5.6 Limites assumées	12
6. Tests unitaires et fonctionnels	12
6.1 Pourquoi tester	12
6.2 Choix du framework : pytest	12
6.3 Organisation des tests	12
6.4 TestClient FastAPI	13
6.5 Résultat des tests	13
6.6 Limites	13
7. Pipeline CI/CD GitHub Actions	13
7.1 Qu'est-ce que la CI/CD	13
7.2 Outil choisi : GitHub Actions	13
7.3 Le fichier tests.yml	14
Bloc 1 — Nom et déclencheurs	14
Bloc 2 — Machine virtuelle	14

Bloc 3 — Service PostgreSQL	14
Bloc 4 — Les étapes du job	14
7.4 Résultats et historique	14
7.5 Limites	15
8. Monitoring	15
8.1 Qu'est-ce que le monitoring	15
8.2 Architecture du monitoring	15
8.3 Les métriques Prometheus	16
8.4 Configuration Prometheus	16
8.5 Dashboard Grafana	16
8.6 Python Logging	17
8.7 Différence entre tests et monitoring	17
8.8 Limites	17
9. MLflow tracking	18
9.1 Présentation rapide de MLflow	18
9.2 Ce que je tracke	18
9.3 L'interface MLflow UI	18
9.4 Comment MLflow a influencé mes décisions	19
9.5 Limites	19
10. Bilan et limites	19
10.1 Récapitulatif	19
10.2 Points forts	19
10.3 Limites assumées	19
10.4 Conclusion personnelle	20

1. Introduction

RouteZone est un système en machine learning qui prédit si un accident de la route est grave ou non. Il repose sur les données BAAC 2022-2024 publiées en open data par le Ministère de l'Intérieur, et sur les temps d'intervention réels des secours (pompiers et SAU) calculés via OSRM. Le modèle retenu est LightGBM.

Le rapport E1 a couvert la donnée, le E2 la veille et le choix technique. Ce rapport E3 détaille la suite logique : le service IA opérationnel. La problématique est la suivante : comment ce modèle entraîné est-il devenu un service réel ?

Le modèle LightGBM ne tourne pas seul. Il s'intègre dans une architecture complète : une API REST sécurisée (authentification par clé API et JWT), une base de données PostgreSQL pour les utilisateurs et l'historique des prédictions, 34 tests automatisés, une chaîne CI/CD GitHub Actions, un monitoring avec Prometheus, Grafana et Python Logging, une application Streamlit pour le frontend, et le tout conteneurisé avec Docker. Le cycle de vie du modèle est tracé avec MLflow.

2. Architecture globale du service IA

2.1 Vue d'ensemble

Mon projet tourne sur 5 containers Docker orchestrés par docker-compose, qui communiquent entre eux :

- **routezone_db** (PostgreSQL 16)
- **routezone_osrm** (moteur de routage)
- **routezone_api** (FastAPI unifiée sur le port 8001)
- **routezone_prometheus** (collecte des métriques)
- **routezone_grafana** (visualisation, port 3000)

Le frontend Streamlit tourne en dehors des containers, en local sur le port 8501.

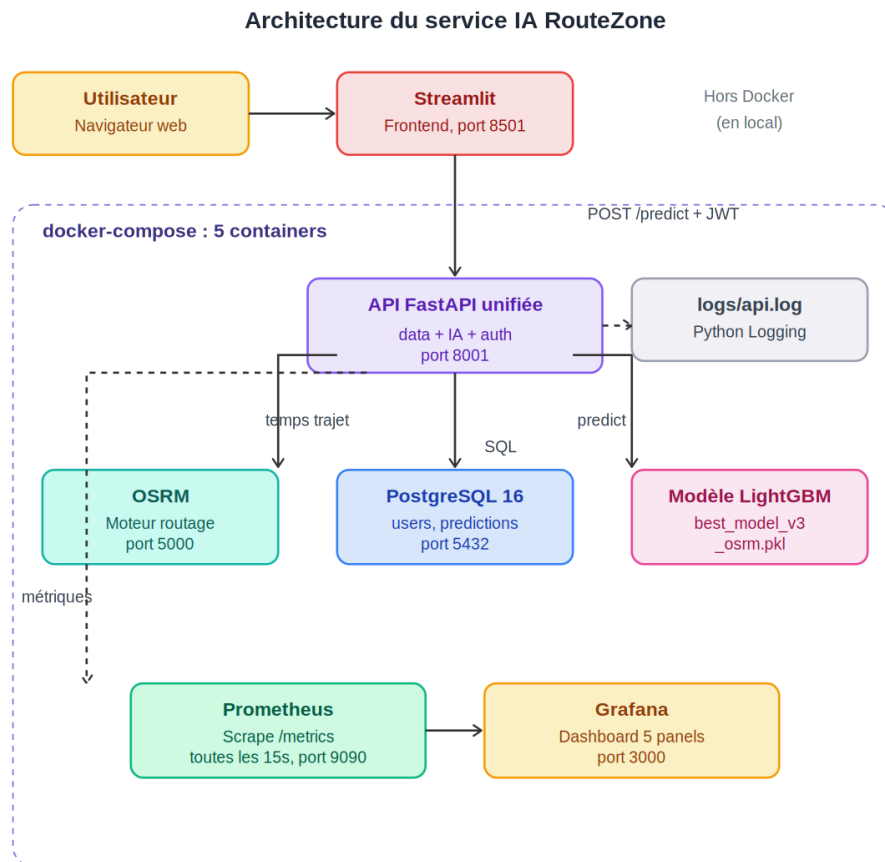


Figure 1 — Architecture du service IA RouteZone

2.2 Flux de données pour une prédiction

- L'utilisateur saisit un accident dans Streamlit en remplissant les différents champs (type de route, âge, luminosité, météo...)
- Streamlit appelle OSRM pour calculer les temps d'intervention réels (pompiers + SAU) et détecter les zones blanches
- Streamlit envoie une requête POST sur /predict de l'API, avec un JWT dans l'en-tête
- L'API vérifie le JWT (via security.py) et valide les données entrantes avec Pydantic

L'API charge le modèle LightGBM (best_model_v3_osrm.pkl). Ce fichier .pkl est une sérialisation du modèle : transformer un objet Python en fichier binaire pour le recharger ensuite. J'utilise joblib, spécialisé pour les modèles ML car plus rapide que pickle standard sur les gros objets numériques.

- L'API lance la prédiction et enregistre le résultat dans PostgreSQL avec l'identifiant de l'utilisateur
- L'API retourne la réponse à Streamlit qui l'affiche (label + probabilité)
- En parallèle, l'API incrémente les compteurs Prometheus et écrit dans logs/api.log

2.3 Endpoints et documentation Swagger

Mon API expose 16 endpoints répartis dans trois routers (auth, data, ia) plus deux endpoints système (/ et /metrics). FastAPI génère automatiquement une page Swagger UI accessible sur /docs (donc <http://localhost:8001/docs>), qui liste tous les endpoints avec leurs paramètres et permet de les tester directement depuis le navigateur, sans avoir besoin d'installer Postman.

2.4 Justification des choix d'architecture

Pourquoi Docker : fiabilité, portabilité et reproductibilité. Le déploiement est simple grâce à docker-compose up.

Pourquoi une API unifiée plutôt que 2 séparées : à l'origine j'avais 2 API distinctes (Data et IA), que j'ai fusionnées. Bénéfices : moins de complexité opérationnelle (un seul container, une seule Swagger), authentification mutualisée, moins de configuration réseau entre containers. Une fonction de routes_ia.py peut appeler une fonction de routes_data.py directement, sans réseau.

Pourquoi PostgreSQL plutôt que SQLite : justifié dans le rapport E1 (section 4). Meilleure gestion de la concurrence, support des types géospatiaux pour la carte Streamlit, conformité ACID.

3. API REST IA — FastAPI

3.1 Pourquoi FastAPI

- L'un des frameworks Python les plus rapides du marché
- Validation automatique des données via Pydantic (sécurité par défaut)
- Documentation Swagger générée automatiquement
- Framework moderne, bien documenté, avec une navigation simple

Ces choix sont détaillés dans le benchmark du rapport E2.

3.2 Architecture interne de l'API

- **1 point d'entrée** : main.py qui crée l'instance FastAPI et monte les routers
- **3 routers** : routes_auth.py, routes_data.py, routes_ia.py
- **4 modules transverses** : security.py, db.py, logger.py, metrics.py

Architecture interne de l'API RouteZone

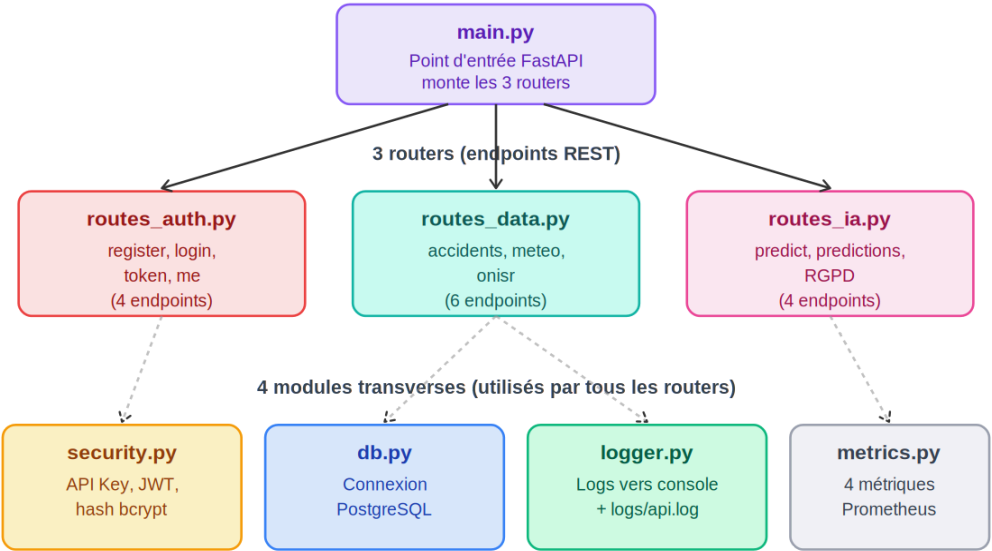


Figure 2 — Architecture interne de l'API RouteZone

3.3 Liste des endpoints

URL	Méthode	Auth	Description
/	GET	Aucune	Health check, infos API
/metrics	GET	Aucune	Métriques Prometheus
/auth/register	POST	Aucune	Créer un compte utilisateur
/auth/login	POST	Aucune	Se connecter (JSON), obtient JWT
/auth/token	POST	Aucune	Se connecter (OAuth2 form-data)
/auth/me	GET	JWT	Infos de l'utilisateur connecté
/accidents	GET	API Key	Liste accidents (filtres)
/accidents/stats	GET	API Key	Statistiques globales
/accidents/departement/{dep}	GET	API Key	Stats d'un département
/accidents/gravite	GET	API Key	Répartition par gravité
/meteo/stats	GET	API Key	Stats météo
/onisr/barometre	GET	API Key	Baromètre ONISR
/predict	POST	API Key OU JWT	Prédire la gravité
/predictions	GET	JWT	Historique des prédictions
/me/predictions	DELETE	JWT	Supprimer (RGPD art. 17)
/me/predictions/export	GET	JWT	Exporter JSON (RGPD art. 20)

3.4 Authentification : API Key + JWT

a) Authentification par clé API

Sert quand un autre logiciel veut utiliser mon API (pas un humain). Le client envoie un header X-API-Key. La comparaison se fait avec `hmac.compare_digest` qui empêche les attaques par mesure de temps : l'opérateur `==` classique s'arrête dès qu'il trouve une différence, plus la clé envoyée ressemble à la vraie, plus la comparaison met de temps. Un attaquant peut deviner la clé caractère par caractère en mesurant le temps de réponse. `hmac.compare_digest` met toujours le même temps quelle que soit l'entrée.

b) Authentification par JWT (utilisateurs humains)

Le protocole HTTP est sans mémoire : chaque requête est indépendante. Mon API ne se souvient pas que l'utilisateur s'est connecté il y a 2 minutes. Donc à chaque requête il faut un moyen de prouver qui on est, sans renvoyer le mot de passe.

JWT = JSON Web Token. C'est une chaîne signée cryptographiquement par mon API avec une clé secrète (JWT_SECRET_KEY dans `.env`). La signature est un code mathématique calculé à partir du contenu + clé secrète. Si quelqu'un modifie le contenu (par exemple change le `user_id` pour devenir admin), la signature ne correspond plus et mon API rejette le token. À chaque requête, l'utilisateur renvoie ce JWT dans le header `Authorization: Bearer <token>`. Mon API vérifie la signature sans avoir besoin de consulter la BDD, ce qui rend le JWT plus performant que les sessions classiques.

Le JWT expire au bout de 24h (configurable dans `.env`). Passé ce délai, l'utilisateur doit se reconnecter.

c) Hashage bcrypt des mots de passe

Les mots de passe ne sont jamais stockés en clair en BDD. Sans cette précaution, en cas de fuite, tous les mots de passe seraient publics.

À l'inscription, `passlib` (avec `bcrypt`) transforme `motdepasse123` en `$2b$12$EixZaYVK1fsbw1ZfbX3OXe....`. Cette transformation est à sens unique : on peut hasher mais pas revenir au mot de passe d'origine.

Important : l'utilisateur tape toujours son mot de passe en clair, c'est mon API qui re-hashe et compare au hash stocké.

J'ai choisi `bcrypt` pour deux raisons : il est lent volontairement (tester des milliards de combinaisons prend des années, contre quelques heures avec MD5), et il ajoute un sel automatique (deux utilisateurs avec le même mot de passe auront deux hashes différents).

3.5 Validation des entrées avec Pydantic

Pydantic valide automatiquement chaque champ envoyé à l'API. Dans mon code, `AccidentInput` définit le format attendu avec des bornes précises : âge entre 0 et 120, heure entre 0 et 23, mois entre 1 et 12, etc. Si les contraintes ne sont pas respectées (par exemple `age=999`), l'API renvoie une erreur HTTP 422 sans même appeler le modèle. Protection efficace contre les données aberrantes.

```
from pydantic import BaseModel, Fieldclass AccidentInput(BaseModel):    age: int = Field(..., ge=0, le=120)    heure: int = Field(..., ge=0, le=23)    mois: int = Field(..., ge=1, le=12)    vma: int = Field(..., ge=0, le=200)    temperature: float = Field(..., ge=-30, le=50)    lat: float = Field(None, ge=-90, le=90)    lon: float = Field(None, ge=-180, le=180)
```

3.6 Gestion des erreurs

Code	Signification	Exemple
200	Tout est en ordre	Prédiction réussie
401	Token JWT invalide ou expiré	Token de plus de 24h
403	Authentification manquante ou invalide	Clé API absente
404	Ressource non trouvée	Département inexistant
409	Conflit	Email déjà utilisé à l'inscription
422	Données invalides	Âge à 224 ans
500	Erreur serveur	Base de données inaccessible

3.7 Conformité RGPD

Le RGPD (Règlement Général sur la Protection des Données) est obligatoire pour toute application traitant des données personnelles. Il repose sur 6 grands principes : collecter uniquement les données strictement nécessaires, garantir la transparence, faciliter l'exercice des droits (suppression, accès, portabilité), fixer une durée de conservation, sécuriser les données, et inscrire la conformité dans une démarche continue.

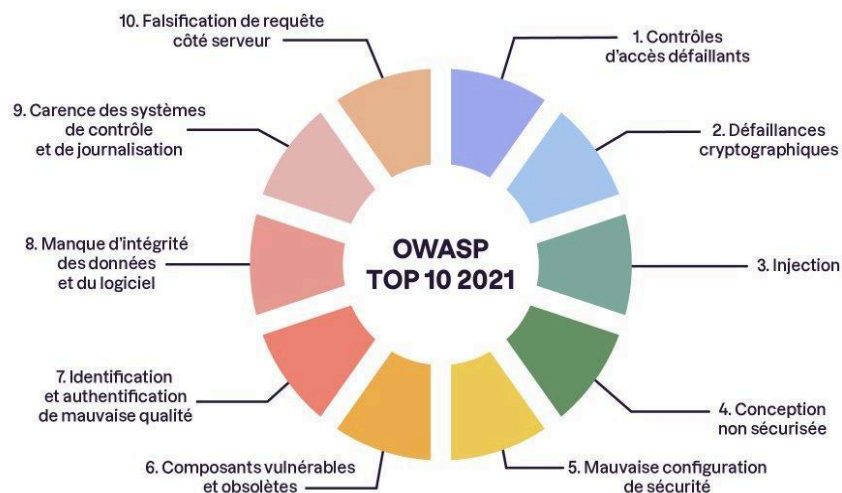
Dans RouteZone, j'ai implémenté deux endpoints dédiés au RGPD :

- **DELETE /me/predictions** — Article 17 (droit à l'effacement). L'utilisateur peut supprimer toutes ses prédictions en une requête.
- **GET /me/predictions/export** — Article 20 (droit à la portabilité). L'utilisateur peut récupérer ses données au format JSON.

Ces deux endpoints sont protégés par JWT : un utilisateur ne peut accéder qu'à ses propres données.

3.8 Sécurité OWASP

L'OWASP (Open Web Application Security Project) est une fondation internationale à but non lucratif qui met à disposition des ressources pour sécuriser les applications web. Tous les 4 ans environ, l'OWASP publie un classement des 10 vulnérabilités les plus critiques pour les applications web : le Top 10 OWASP.



Les 10 familles de vulnérabilité référencées par l'OWASP en 2021

Figure 3 — Le Top 10 OWASP 2021 (source : OpenClassrooms)

Voici les points OWASP que j'ai adressés dans RouteZone :

Vulnérabilité OWASP	Mesure mise en place dans RouteZone
A01 — Contrôles d'accès défaillants	Chaque utilisateur ne peut accéder qu'à ses propres prédictions grâce au filtre SQL WHERE user_id = :uid dans tous les endpoints /me/* et /predictions
A02 — Défaillances cryptographiques	Mots de passe hashés avec bcrypt, JWT signé avec l'algorithme HS256
A03 — Injection	Toutes les requêtes SQL utilisent SQLAlchemy avec paramètres bindés. Le texte SQL et les valeurs sont envoyés séparément à la BDD : la valeur reçue est traitée comme un texte, pas comme du code SQL à exécuter.
A07 — Identification de mauvaise qualité	JWT avec expiration à 24h, hmac.compare_digest, hashage bcrypt
A09 — Carence de journalisation	Toutes les requêtes IA et les actions RGPD sont tracées dans logs/api.log via le module logger.py

Limites assumées :

- **Pas de rate limiting** : un utilisateur peut envoyer un nombre illimité de requêtes. Un attaquant peut essayer des milliers de mots de passe par minute sur /auth/login (force brute) ou saturer le serveur (DoS). Amélioration prévue : slowapi.
- **Pas de HTTPS en local** : l'API tourne en HTTP. En HTTP, les données circulent en clair : un attaquant sur un wifi public pourrait lire les mots de passe et JWT. En production il faudrait un reverse proxy (nginx, Traefik) avec certificat SSL/TLS.

4. Modèle ML servi (LightGBM)

4.1 Rappel rapide du modèle

Le modèle retenu est LightGBM version 3 sans calibration, pour une classification binaire Grave / Pas grave.

Métrique	Valeur
Recall GRAVE	0,7643
Precision GRAVE	0,4166
F1 macro	0,6956
AUC-ROC	0,8558
Accuracy	0,7760

Pour plus de détails sur le choix de ce modèle, voir le rapport E2 (section 6).

4.2 Chargement du modèle

Le modèle est chargé une seule fois au démarrage de l'API, pas à chaque requête :

```
model = joblib.load(MODELS_DIR / "best_model_v3_osrm.pkl")
features = joblib.load(MODELS_DIR / "features_v3_osrm.pkl")
class_mapping = joblib.load(MODELS_DIR / "class_mapping.pkl")
```

- **best_model_v3_osrm.pkl** : le modèle entraîné
- **features_v3_osrm.pkl** : la liste des 34 features dans l'ordre attendu
- **class_mapping.pkl** : la correspondance sorties modèle (0/1) ↔ labels lisibles (Pas grave/Grave)

4.3 Stratégie multi-versions

Mon code charge automatiquement la meilleure version disponible :

```
if v3_path.exists():    model = joblib.load(v3_path)        # priorite V3
elif v2_path.exists(): model = joblib.load(v2_path)        # fallback V2
else:                  model = joblib.load("best_model.pkl") # fallback V1
```

Trois raisons : traçabilité complète V1 → V2 → V3, fallback automatique si une version échoue, comparaison locale possible.

4.4 Pipeline de prédiction

- Réception des données utilisateur (validées par Pydantic)
- Calcul des features dérivées (créneau, tranche d'âge, jour de la semaine, etc.)
- Construction du DataFrame dans l'ordre des features attendus
- Conversion des features catégorielles en type category (format LightGBM)
- Prédiction avec model.predict_proba(X)
- Application du seuil 0,5 : probabilité ≥ 0,5 → Grave, sinon Pas grave
- Retour de la réponse JSON

Pipeline de prédiction de l'endpoint /predict

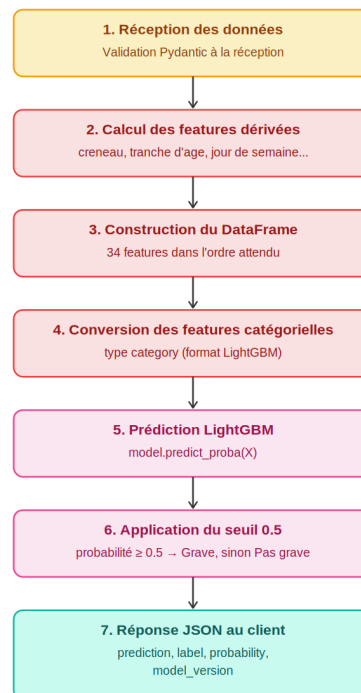


Figure 4 — Pipeline de prédiction de l'endpoint /predict

4.5 Décision technique : calibrator désactivé

Le calibrage isotonique (scikit-learn) est censé rendre les probabilités plus interprétables. Dans mon cas, je ne peux pas l'utiliser car il dégrade le Recall sur la classe Grave (passe de 76 % à 33 %).

Concrètement, si le calibrator restait activé, il classerait à tort trop d'accidents graves comme Pas grave, ce qui n'est pas tolérable pour un système d'aide aux services de secours. Je l'ai donc désactivé. J'ai conservé le fichier .pkl du calibrator pour la traçabilité. La démarche d'investigation est documentée dans docs/incidents/incident_calibrator.md.

4.6 Performances en production

Temps de prédiction moyen < 100 ms. Le test `test_predict_temps_reponseraisonnable` valide < 2s. Le test `test_predict_reproductibilite` garantit qu'une même entrée produit toujours la même prédiction.

5. Application prototype Streamlit

5.1 Présentation de Streamlit

Streamlit est une bibliothèque Python open source qui permet de créer des applications web en quelques lignes de code. Avantages : développement rapide, pas besoin de HTML/CSS/JS, idéal pour les apps data et ML. Limite : personnalisation visuelle moins fine que React.

5.2 Utilisateurs cibles

- Services de secours : pour prioriser les interventions sur les cas potentiellement graves
- Compagnies d'assurance : pour évaluer la gravité probable d'un sinistre
- Collectivités : pour identifier les zones à risque
- Étudiants et chercheurs : pour analyser les données BAAC

5.3 Parcours utilisateur

Mon application est organisée en 4 pages principales :

- Page de connexion / inscription
- Page de prédiction (formulaire, résultat, carte interactive)
- Page Historique des prédictions
- Page Mes données RGPD (export JSON + suppression)

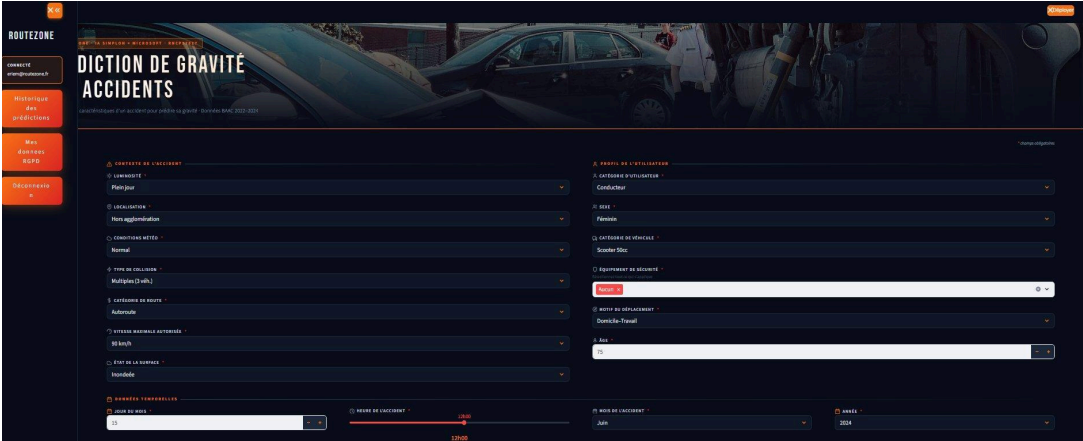


Figure 5 — Formulaire de prédiction Streamlit

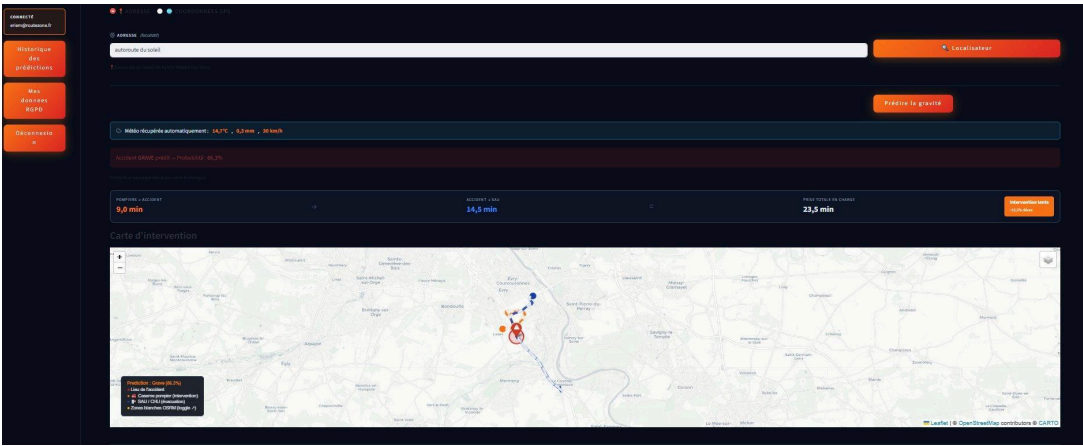


Figure 6 — Résultat d'une prédiction avec carte interactive

Historique de vos prédictions

PRÉDICTIONS TOTALES: 15

TOMBES: 11

PROBABILITÉ MOYENNE: 65,7%

Date	Label	Probabilité (%)	Age	Sexe	Hauteur	Poids	Temps
2024-05-20T14:26	Grave	95,3	75	12	180	80,000	10
2024-05-20T14:26	Grave	77,7	85	12	180	80,000	10
2024-05-20T14:26	Grave	77,7	85	12	180	80,000	10
2024-05-20T14:26	Grave	77,7	85	12	180	80,000	10
2024-05-20T14:26	Grave	85,4	85	12	180	80,000	10
2024-05-20T14:26	Grave	82,6	85	12	180	80,000	10
2024-05-20T14:26	Grave	80,2	85	12	180	80,000	10
2024-05-14T13:35	Grave	90,9	38	3	180	80,000	10
2024-05-14T13:34	Grave	90,2	38	3	180	80,000	10
2024-05-12T10:00	Peu grave	49,6	35	12	180	80,000	10

Retour aux prédictions

Figure 7 — Page Historique des prédictions

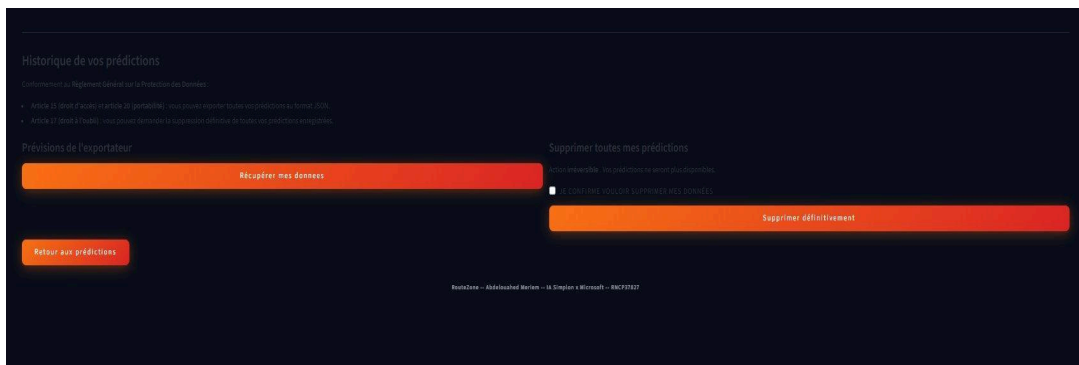


Figure 8 — Page Mes données RGPD

5.4 Communication entre Streamlit et l'API

- Streamlit utilise la bibliothèque Python requests pour envoyer des requêtes HTTP à l'API
- Le JWT est stocké dans `st.session_state` après le login
- À chaque appel API, le JWT est envoyé dans le header `Authorization: Bearer <token>`
- Si le token est expiré (24h), l'API renvoie 401 et Streamlit demande à se reconnecter

5.5 Design et identité visuelle

Palette : Orange #F97316 (signature RouteZone), Bleu nuit #0A0E1A (fond), Gris foncé #111827 (cartes). Accents rouge #EF4444 et vert #22C55E (feu tricolore).

Typographies : Bebas Neue (titres) et DM Sans (corps), via Google Fonts.

Mode sombre par défaut : réduit la fatigue visuelle pour les services de secours en horaires variés, met en valeur l'orange, consomme moins d'énergie sur écrans OLED.

Icônes SVG plutôt qu'emojis : net à toute taille, contrôle CSS précis, plus léger qu'un PNG, rendu identique sur tous les navigateurs (contrairement aux emojis qui changent entre Apple, Google et Microsoft).

5.6 Limites assumées

- **Pas de tests automatisés sur Streamlit** : Streamlit est difficile à tester automatiquement (re-exécution complète du script à chaque interaction). Compensé par 34 tests pytest sur l'API.
- **Pas de fonction "mot de passe oublié"** : amélioration prévue avec email + lien de réinitialisation
- **Personnalisation visuelle limitée** par les composants natifs Streamlit

6. Tests unitaires et fonctionnels

6.1 Pourquoi tester

- **Détecter les régressions** : être sûr qu'une modification ne casse pas l'existant
- **Garantir la fiabilité** : un bug coûte cher sur un outil d'aide aux secours
- **Déployer en confiance** : tests verts = push serein
- **Documentation vivante** : un test bien nommé décrit le comportement attendu

6.2 Choix du framework : pytest

Pytest est le framework de test le plus utilisé dans l'écosystème Python. Syntaxe simple (fonctions `test_*`), bonne intégration FastAPI via TestClient.

Distinction pytest et CI/CD : pytest est l'outil qui exécute les tests. La CI/CD (section 7) est l'automatisation qui déclenche pytest à chaque push. En local, j'utilise pytest seul. En CI, GitHub Actions orchestre pytest.

6.3 Organisation des tests

34 tests répartis dans deux fichiers :

Famille	Nombre	Fichier
Authentification	4	test_api.py
Endpoints data	8	test_api.py
Endpoints IA	6	test_api.py
Smoke tests modèle	5	test_api.py
Bornes Pydantic	5	test_api.py
Cohérence métier	5	test_business_logic.py
Health check	1	test_api.py
Total	34	

Focus sur les tests de cohérence métier (point fort) : 5 tests qui vérifient que le modèle a appris la logique métier, pas juste des corrélations statistiques :

- test_frontale_plus_grave_que_arriere
- test_aucun_equipement_plus_grave_que_ceinture
- test_nuit_plus_grave_que_jour
- test_moto_plus_grave_que_voiture
- test_vma_haute_plus_grave_que_vma_basse

Si l'un échoue, c'est que le modèle a perdu une logique fondamentale, à investiguer avant tout déploiement.

6.4 TestClient FastAPI

Pour tester l'API, je n'ai pas besoin de la démarrer avec uvicorn. TestClient simule les requêtes HTTP en interne, dans le même processus Python. Tests rapides, isolés, portables (locaux ou CI).

```
from fastapi.testclient import TestClientfrom main import appclient = TestClient(app)response = client.get("/", headers={"X-API-Key": "routezone-secret-2024"})assert response.status_code == 200
```

6.5 Résultat des tests

- 34 tests sur 34 verts (100 % de réussite)
- Temps d'exécution : environ 26 secondes en local
- Exécutés à chaque push sur master via GitHub Actions



Figure 9 — Résultat de pytest en local : 34 tests collected



Figure 10 — Badge "passing" dans le README

6.6 Limites

- **Pas de tests unitaires purs** : tous mes tests passent par TestClient (donc tests d'intégration)
- **Couverture pytest-cov non mesurée** : amélioration prévue, viser 70-80 %
- **Pas de tests de charge** : Locust ou k6 à intégrer pour la résistance à la montée en charge

7. Pipeline CI/CD GitHub Actions

7.1 Qu'est-ce que la CI/CD

La CI/CD (Continuous Integration / Continuous Delivery) automatise les phases de test, intégration et déploiement.

Dans mon projet, j'ai implémenté la partie CI : à chaque push sur GitHub, mes 34 tests sont lancés automatiquement. Le CD (déploiement automatique) n'est pas implémenté, amélioration prévue.

7.2 Outil choisi : GitHub Actions

- Gratuit pour les dépôts publics et comptes personnels
- Intégré nativement à GitHub où vit mon code
- Configuration simple en YAML
- Catalogue d'actions prêtes à l'emploi
- Standard du marché en 2026

Alternative	Raison de ne pas choisir
Jenkins	Demande un serveur dédié à installer et maintenir. Surdimensionné pour un POC.
GitLab CI	Mon code est sur GitHub, pas GitLab
CircleCI	Quota gratuit limité, moins intégré
Travis CI	A perdu du terrain depuis 2020

7.3 Le fichier tests.yml

Le fichier tests.yml est un manuel qui dicte à GitHub Actions ce qu'il doit faire à chaque push. Il se situe dans .github/workflows/tests.yml. GitHub Actions cherche automatiquement les fichiers de config dans ce dossier.

Bloc 1 — Nom et déclencheurs

```
name: Tests RouteZoneon: push: branches: [master] workflow_dispatch:
```

Bloc 2 — Machine virtuelle

```
jobs: test: runs-on: ubuntu-latest
```

GitHub démarre une VM Ubuntu vierge pour exécuter le job. Cette VM est jetable : créée puis détruite après chaque run.

Bloc 3 — Service PostgreSQL

```
services: postgres: image: postgres:16-alpine env: POSTGRES_USER: routezone
POSTGRES_PASSWORD: routezone_pwd_2024 POSTGRES_DB: routezone ports: - 5432:5432
options: --health-cmd pg_isready
```

J'utilise la même version de PostgreSQL en local et en CI, ce qui garantit des conditions identiques.

Bloc 4 — Les étapes du job

```
steps: - name: Checkout code uses: actions/checkout@v4 - name: Setup Python uses:
actions/setup-python@v5 with: python-version: '3.11' cache: 'pip' - name: Install
dependencies run: pip install -r requirements.txt - name: Import data run: python
bdd/import_data.py - name: Run pytest run: pytest tests/ -v --tb=short
```

7.4 Résultats et historique

- 17 workflow runs réalisés au total
- 14 verts (succès), 3 rouges (échecs corrigés en début de projet)
- Durée moyenne : 2-3 minutes par run

Les 3 échecs initiaux correspondent à des moments où mon code n'était pas encore stabilisé. Ces erreurs m'ont été utiles : prévenue immédiatement, j'ai pu corriger avant que le bug ne se propage.

All workflows

Showing runs from all workflows

Filter workflow runs

17 workflow runs

Workflow

Event

Status

Branch

Actor

✓

rapport e1/e2 complet

Tests RouteZone #17: Commit 97a7962 pushed by Meriem69

master

📅

19 mai, 18:44 UTC+2

⌚

2m 53s

...

✓

maj

Tests RouteZone #16: Commit 81695e1 pushed by Meriem69

master

📅

May 15, 3:53 PM GMT+2

⌚

2m 43s

...

✓

docs(e5): incident calibrator - documentation pedagogique

Tests RouteZone #15: Commit c6c859c pushed by Meriem69

master

📅

15 mai, 14:25 UTC+2

⌚

3m 26s

...

✓

docs: MAJ readme

Tests RouteZone #14: Commit e58eeed pushed by Meriem69

master

📅

14 mai, 22:41 UTC+2

⌚

2m 41s

...

✓

fix: desactivation calibrator + bugs UI Streamlit + carnet veille E2

Tests RouteZone #13: Commit 02912e6 pushed by Meriem69

master

📅

14 mai, 20:40 UTC+2

⌚

3m 6s

...

✓

fix: desactivation calibrator dans flux predict + carnet de bord veille

Tests RouteZone #12: Commit 2a88b2a pushed by Meriem69

master

📅

14 mai, 16:10 UTC+2

⌚

3m 20s

...

✗

chore: nettoyage architecture + gitignore complet

Tests RouteZone #11: Commit b4d6da8 pushed by Meriem69

master

📅

12 mai, 23:41 UTC+2

⌚

2m 52s

...

✗

chore

Tests RouteZone #10: Commit 6d9c297 pushed by Meriem69

master

📅

12 mai, 22:06 UTC+2

⌚

2m 52s

...

✗

feat: champ annee meteo + tests coherence metier + UX accessibilite

Tests RouteZone #9: Commit 8a77ba6 pushed by Meriem69

master

📅

12 mai, 20:43 UTC+2

⌚

2m 43s

...

✓

feat: RGPD endpoints + Streamlit refonte UI (VMA, surface, meteo auto)

Tests RouteZone #8: Commit 74f11ba pushed by Meriem69

master

📅

12 mai, 14:25 UTC+2

⌚

2m 50s

...

✓

feat(rgpd): endpoints DELETE et GET /me/predictions + /auth/token OAuth2

Tests RouteZone #7: Commit 19a2143 pushed by Meriem69

master

📅

12 mai, 11:11 UTC+2

⌚

2m 37s

...

✓

feat(monitoring): ajout container Grafana + dashboard 5 panels

Tests RouteZone #6: Commit 86177d8 pushed by Meriem69

master

📅

10 mai, 19:56 UTC+2

⌚

2m 45s

...

✓

feat (monitoring) : metrique Prometheus + container + scrape config

Tests RouteZone #5: Commit 6600c1 pushed by Meriem69

master

📅

10 mai, 16:26 UTC+2

⌚

2m 52s

...

✓

monitoring : logging structure dans logs/api.log

Tests RouteZone #4: Commit 14ba33e pushed by Meriem69

master

📅

10 mai, 15:01 UTC+2

⌚

2m 50s

...

✓

test: ajout 10 tests smoke ML + bornes Pydantic (29 tests au total)

Tests RouteZone #3: Commit eba0194e pushed by Meriem69

master

📅

9 mai, 18:37 UTC+2

⌚

2m 47s

...

✓

ajout badge CI/CD au README

Tests RouteZone #2: Commit d410477 pushed by Meriem69

master

📅

9 mai, 17:40 UTC+2

⌚

2m 51s

...

✓

ajout workflow pr test

Tests RouteZone #1: Commit 8614ab7 pushed by Meriem69

master

📅

9 mai, 17:28 UTC+2

⌚

2m 57s

...

Figure 11 — Historique des workflow runs GitHub Actions (17 runs)

7.5 Limites

- **Pas encore de CD** : aujourd'hui le workflow ne fait que tester. Une vraie chaîne MLOps déploierait l'image Docker sur un VPS après validation. J'ai choisi de me concentrer sur la CI pour garantir la qualité du code, le CD viendra ensuite.
- **Pas de build Docker dans la CI** : pourrait pousser sur Docker Hub ou GHCR après tests verts
- **Pas de scan de sécurité automatique** : Snyk ou Dependabot à intégrer

8. Monitoring

8.1 Qu'est-ce que le monitoring

Le monitoring est un ensemble de techniques pour analyser, contrôler et surveiller une application en temps réel. Indispensable en informatique : mesure la qualité et la fiabilité des logiciels, réseaux, serveurs.

Même si tous mes tests sont verts, je dois surveiller l'application après son démarrage pour détecter rapidement les anomalies. Sans monitoring, je découvrirais les bugs quand un utilisateur viendrait se plaindre, trop tard.

Sur un projet ML, le monitoring sert aussi à détecter le drift du modèle : la dégradation progressive des prédictions dans le temps. Si les données réelles changent (nouveaux véhicules, nouvelles infrastructures), le modèle peut perdre en précision sans que ça apparaisse dans les métriques classiques.

8.2 Architecture du monitoring

Pilier	Outil	Rôle
Métriques	Prometheus	Compte ce qui se passe (nombre de prédictions, latence, erreurs)
Visualisation	Grafana	Affiche les métriques sous forme de graphiques temps réel
Logs	Python Logging	Enregistre chronologiquement les événements dans un fichier

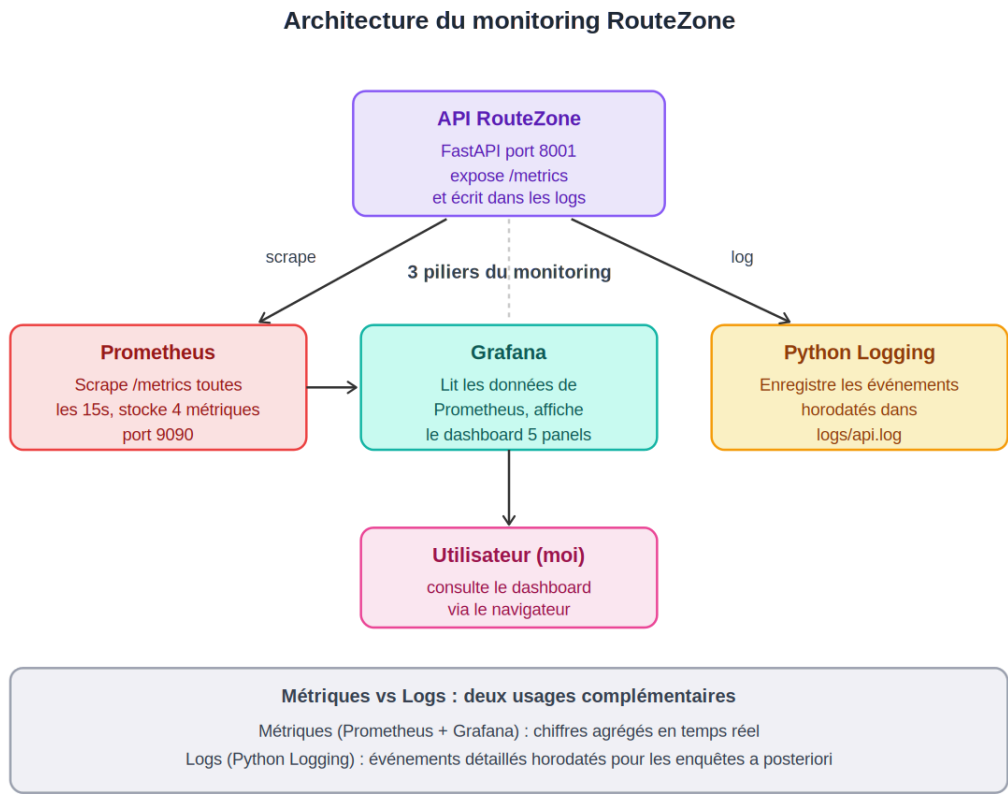


Figure 12 — Architecture du monitoring RouteZone

8.3 Les métriques Prometheus

4 métriques définies dans metrics.py :

Métrique	Type	Description
routezone_predictions_total	Counter	Nombre total de prédictions
routezone_predictions_par_classe_total	Counter	Par classe (Grave / Pas grave)
routezone_prediction_duration_seconds	Histogram	Latence des prédictions
routezone_errors_total	Counter	Nombre total d'erreurs

Counter : compteur qui ne fait qu'augmenter (nombre total de prédictions depuis le démarrage).

Histogram : mesure une distribution (combien de prédictions sous 0,1 seconde, entre 0,1 et 0,2 seconde, etc.).

8.4 Configuration Prometheus

```
global:
  scrape_interval: 15s
  scrape_configs:
    - job_name: 'routezone_api'
      static_configs:
        - targets: ['host.docker.internal:8001']
      metrics_path: '/metrics'
```


Prometheus interroge l'API toutes les 15 secondes pour récupérer les métriques. À chaque scrape, il stocke les valeurs avec un horodatage, ce qui permet de tracer leur évolution dans le temps.

8.5 Dashboard Grafana

Mon dashboard contient 5 panels qui couvrent les métriques essentielles : nombre total de prédictions, prédictions par classe, latence moyenne sur 5 minutes, prédictions par minute, total des erreurs.

- Grafana lit Prometheus, il ne stocke rien lui-même
- Rafraîchissement temps réel (toutes les 5 à 15 secondes)
- Accessible sur <http://localhost:3000>

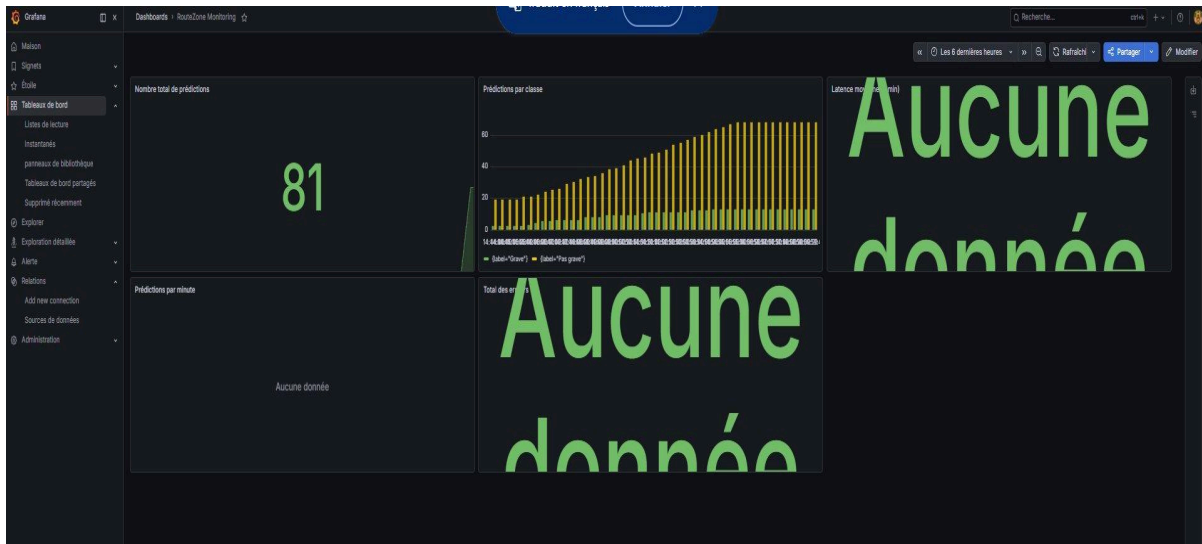


Figure 13 — Dashboard Grafana RouteZone (81 prédictions enregistrées)

La capture ci-dessus montre le dashboard en cours de session de test. On observe 81 prédictions enregistrées et un bar chart Grave/Pas grave qui montre la répartition. Les panels Latence moyenne, Prédictions par minute et Total des erreurs affichent "Aucune donnée" car ces panels utilisent des requêtes Prometheus qui nécessitent un historique suffisant pour calculer un rate. En usage continu sur plusieurs heures, ces panels affichent leurs valeurs respectives.

8.6 Python Logging

Le module `logger.py` écrit dans la console et dans le fichier `logs/api.log`. Format : 2026-05-10 13:52:14 | INFO | routezone | message. Niveaux utilisés : INFO (événements normaux), WARNING et ERROR (anomalies). Le fichier est persistant : l'historique est conservé même après redémarrage de l'API.

```
2026-05-13 14:23:18 | INFO | routezone | Prediction demandee : age=35, vma=80
2026-05-13 14:23:18 | INFO | routezone | Prediction terminee : label=Pas grave, proba=23.4%
2026-05-13 14:25:42 | INFO | routezone | RGPD : suppression predictions pour user_id=3
```

Métriques et logs : une métrique est un calcul agrégé ("48 prédictions dans les 5 dernières minutes"). Un log est une trace détaillée et horodatée d'un événement précis. Les deux sont complémentaires : les métriques donnent une vue d'ensemble, les logs permettent l'enquête après un incident.

8.7 Différence entre tests et monitoring

Les tests vérifient que le code fonctionne avant la mise en ligne. Test échoue, je corrige avant de déployer.

Le monitoring surveille ce qui se passe après la mise en ligne, en conditions réelles. Anomalie, je suis prévenue immédiatement.

Aucun des deux ne remplace l'autre : les tests préviennent les bugs connus à l'avance, le monitoring détecte les problèmes imprévus à l'usage.

8.8 Limites

- **Pas d'alertes automatiques** : aujourd'hui je dois regarder Grafana manuellement. Alertmanager + Slack/email à configurer.
- **Monitoring en local uniquement** : en production, services externalisés (Grafana Cloud, Datadog) à accessibles partout
- **Pas de monitoring du drift modèle** : Evidently AI ou MLflow Model Monitoring à ajouter
- **Dashboard non versionné** : exporter le JSON dans monitoring/grafana/dashboards/ pour la reproductibilité

9. MLflow tracking

9.1 Présentation rapide de MLflow

MLflow est une plateforme open source créée par Databricks pour gérer le cycle de vie d'un modèle ML. 4 modules :

- **MLflow Tracking** : suivre les expérimentations
- **MLflow Projects** : packaging du code reproductible
- **MLflow Models** : déploiement standardisé
- **MLflow Registry** : gestion formelle des versions en production

Dans RouteZone, j'utilise principalement MLflow Tracking. Détaillé dans le rapport E2 (encadré 3).

9.2 Ce que je tracke

Catégorie	Exemples
Paramètres (hyperparamètres)	max_depth=6, learning_rate=0.05, num_leaves=31, class_weight=balanced
Métriques	Recall GRAVE=0,7643, F1 macro=0,6956, AUC-ROC=0,8558
Artefacts	Modèle (.pkl), matrice de confusion, feature importance
Métadonnées	Nom du run, date, durée, source du notebook

```
import mlflow
mlflow.set_experiment("routezone_v3_osrm")
with mlflow.start_run(run_name="v3_osrm_class_weight_balanced"):
    mlflow.log_params(params_v3)
    mlflow.log_metrics({
        "recall_grave": 0.7643,
        "precision_grave": 0.4166,
        "f1_macro": 0.6956,
        "auc_roc": 0.8558
    })
    mlflow.sklearn.log_model(model_v3, "model")
```

Le mot-clé `with` garantit que le run sera proprement fermé à la fin du bloc, même si le code plante. `mlflow.log_params` enregistre tous les hyperparamètres en un appel. `mlflow.log_metrics` enregistre les métriques sous forme de dictionnaire. `mlflow.sklearn.log_model` sauvegarde le modèle comme artefact attaché au run.

9.3 L'interface MLflow UI

Pour accéder à MLflow : commande `mlflow ui` depuis le terminal (dans notebooks/), accessible sur `http://localhost:5000`. L'interface permet de voir les runs côte à côte, filtrer par métrique, comparer plusieurs runs, télécharger le modèle entraîné.

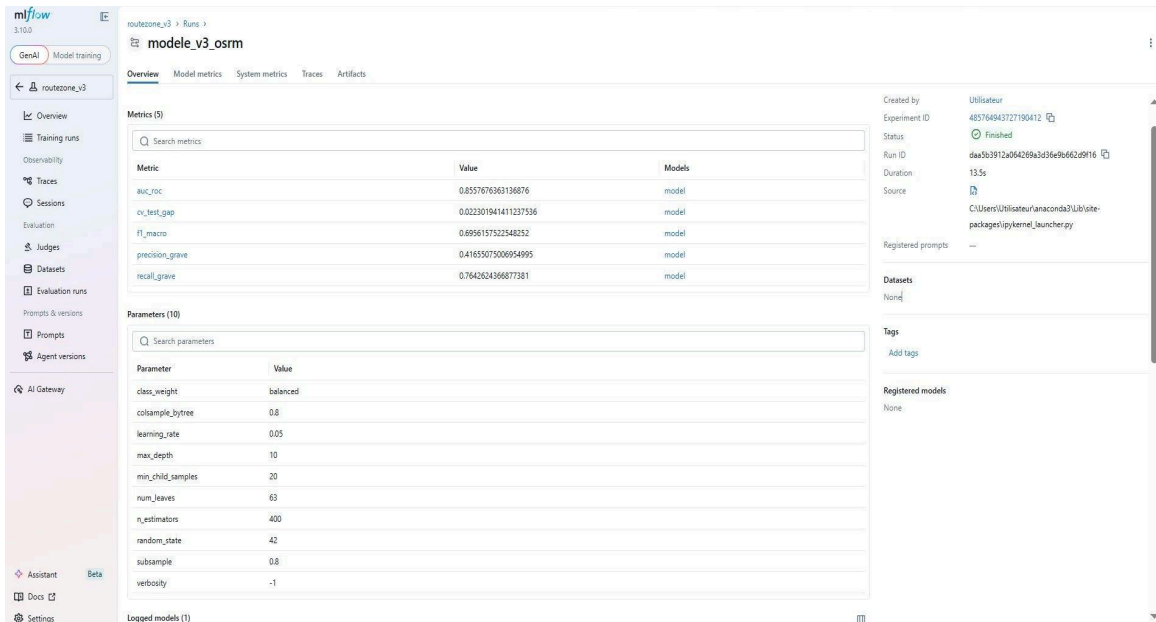


Figure 14 — Détail d'un run MLflow pour le modèle V3 OSRM

La capture ci-dessus montre le détail d'un run pour V3 OSRM : 5 métriques d'évaluation, 10 hyperparamètres, métadonnées (date, durée, source). Statut Finished confirme l'exécution sans erreur.

9.4 Comment MLflow a influencé mes décisions

Sans MLflow j'aurais perdu le fil de mes expérimentations. J'ai testé plus de 10 configurations. MLflow m'a permis de :

- **Comparer V1, V2, V3 OSRM côte à côte** : c'est ce qui a justifié le choix de V3 OSRM (meilleur Recall GRAVE)
- **Abandonner Optuna** : gains marginaux par rapport à la complexité ajoutée, décision évidente en comparant les runs
- **Documenter l'incident calibrator** : constat objectif de la chute du Recall à 33% avec le calibrator activé

9.5 Limites

- **Pas de MLflow Registry** : aujourd'hui uniquement Tracking. Le Registry permettrait de gérer les étapes Staging/Production/Archived
- **Local uniquement** : en production, serveur MLflow centralisé accessible à l'équipe
- **Pas de tracking continu en CI** : intégrer MLflow dans la CI/CD pour un run automatique à chaque réentraînement
- **mlruns/ non versionné** : trop volumineux pour Git. Solution : MLflow distant avec stockage S3

10. Bilan et limites

10.1 Récapitulatif

Dans ce rapport E3, j'ai présenté la mise en service complète du modèle LightGBM RouteZone. L'architecture repose sur 5 containers Docker orchestrés par docker-compose : une API REST FastAPI unifiée (16 endpoints sécurisés par clé API et JWT), une base PostgreSQL pour les utilisateurs et l'historique, un moteur OSRM, et une chaîne de monitoring complète (Prometheus + Grafana + Logging).

L'application est utilisable via Streamlit, testée par 34 tests pytest, et chaque push sur GitHub déclenche une chaîne CI avec GitHub Actions. Le cycle de vie du modèle est tracé avec MLflow.

10.2 Points forts

- **Sécurité** : 5 mesures OWASP adressées (A01, A02, A03, A07, A09)
- **Conformité RGPD** : 2 endpoints dédiés (articles 17 et 20)
- **Cohérence métier validée** : 5 tests spécifiques (moto > voiture, frontale > arrière, etc.)
- **Démarche scientifique** : décisions documentées (calibrator désactivé, Optuna abandonné)
- **Reproductibilité** : projet entièrement portable via Docker

10.3 Limites assumées

Limite	Section	Amélioration prévue
Pas de CD	7.5	Job déploiement Docker sur VPS
Pas de HTTPS local	3.8	Reverse proxy nginx + SSL
Pas de rate limiting	3.8	Intégration slowapi
Pas de tests Streamlit	5.6	Tests manuels uniquement
Pas de pytest-cov	6.6	Intégration pytest-cov
Pas d'alertes Alertmanager	8.8	Alertmanager + Slack
Pas de monitoring drift	8.8	Evidently AI
Pas de MLflow Registry	9.5	Mise en place du Registry
Pas de "mot de passe oublié"	5.6	Email + lien de réinitialisation

10.4 Conclusion personnelle

Ce projet RouteZone m'a permis de mettre en pratique tout ce que la formation Simplon m'a apporté : du choix d'une architecture technique à l'industrialisation d'un modèle de machine learning. J'ai assumé chaque décision, qu'elle soit positive (le choix de FastAPI, la fusion des API, la désactivation du calibrator) ou négative (l'absence de CD, l'absence de monitoring du drift).

J'ai également appris que le choix de l'outil est crucial. Je préfère utiliser un outil parce qu'il répond à un vrai besoin métier, plutôt que par effet de mode. C'est ce raisonnement qui m'a guidée tout au long du projet : FastAPI parce qu'il est rapide et bien adapté à un service IA, LightGBM parce qu'il excelle sur les données tabulaires, MLflow parce que je voulais une traçabilité rigoureuse de mes expérimentations.

Pour les prochaines étapes, je souhaiterais déployer l'API sur un VPS pour passer à un vrai contexte de production, et compléter ma chaîne CI avec un véritable CD automatisé.